question 0:

in this https://huggingface.co/spaces/weege007/ultrascale-playbook, don't understabd context parallel, please use an example, do a step by step calculation, state
1) what is a ring attention,
2) how does ring attention solve the computation and communication problem, what is the benefit, what are the differences between context and sequence parallel,
3) I don't understabd the example in the attached figure, please explain it
4) computation on each gpu,
5) communications among gpus,
6) including forward propagation and backward propagation analysis,
7) how to do the communication overlap
8) comparing different approach, e.g., original ring attention and zig-zag attention
9) comparing different implementation, e.g., all gather implementation and all-to-all (ring implementation)
10) using the similar examples, styles, and details as the examples you did for tensor papallel
11) using formulas, tables, diagram, matrix with values, do the real computation step by step
12) generate a file called context_attention_v0.txt

question 1:
this part is not clear, could you please use detailed examples, do the calculation step by step, similar as the forward step, to clarify the process of backward propogation, thanks

Backward Ring Steps:
Initial: Each GPU has dL/dO_i for its token
Step 0: Compute local gradients
GPU1: Has dL/dO1, Q1, recompute with K1,V1
→ partial dL/dQ1, dL/dK1, dL/dV1
Step 1-3: Ring rotation (counter-clockwise)
Pass K,V and accumulate dL/dK, dL/dV
Each GPU accumulates gradients from all Qs that attended to its K,V
Final: Each GPU has complete dL/dQ_i, dL/dK_i, dL/dV_i for its token
Backward Communication:
- dL/dK and dL/dV must be ACCUMULATED (summed) from all GPUs
- Ring backward: accumulate as K,V rotate back
- Or: use REDUCE-SCATTER at the end

question 2:
in TWO STRATEGIES FOR BACKWARD RING ATTENTION, strategy B,
is this understanding correct?
1. to get
dL/dQ1 = accumulated from K1, K2, K3, K4 (all K chunks seen)
all K, V need to be transferred clockwise

2. to get
dL/dK1 = accumulated from Q1, Q2, Q3, Q4 (all Q contributions)
dL/dV1 = accumulated from Q1, Q2, Q3, Q4 (all Q contributions)
all Q_j, dL/dO_j need to be transferred COUNTER-CLOCKWISE (opposite of forward)

3. conclusion,
all all Q_j, dL/dO_j need to be transferred COUNTER-CLOCKWISE (opposite of forward)
all K, V need to be transferred clockwise

which means all Q, K, V, dL/dO_j need to be transferred, correct?

here:
```
================================================================================
SECTION 9.7: BACKWARD WITH Q,dO ROTATION (Strategy B)
================================================================================


================================================================================
SECTION 9.8: FINAL BACKWARD STATE
================================================================================

 GPU1   | dL/dQ1 = accumulated from K1, K2, K3, K4 (all K chunks seen)       |
|        | dL/dK1 = accumulated from Q1, Q2, Q3, Q4 (all Q contributions)     |
|        | dL/dV1 = accumulated from Q1, Q2, Q3, Q4 (all Q contributions)     |


================================================================================================
================================================================
```

Answers:

```
================================================================================
Context Parallelism & Ring Attention - Comprehensive Guide
================================================================================

This document explains Context Parallelism (CP), specifically Ring Attention,
with detailed step-by-step calculations, similar to our Tensor Parallelism examples.

================================================================================
SECTION 1: WHAT IS RING ATTENTION?
================================================================================

Ring Attention is a technique for distributing ATTENTION computation across
multiple GPUs by splitting the SEQUENCE dimension, allowing training with
extremely long sequences that don't fit in a single GPU's memory.

┌──────────────────────────────────────────────────────────────────────────────┐
│ CORE IDEA                                                                    │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│ Problem: Self-attention has O(s²) memory complexity                          │
│          For sequence length s = 128K, attention matrix = 128K × 128K       │
│          = 16 billion elements = 64GB just for attention scores!             │
│                                                                              │
│ Solution: Split sequence across GPUs, compute attention in CHUNKS            │
│           Pass K,V around a ring while keeping Q local                       │
│           Never materialize full attention matrix                            │
│                                                                              │
```

Standard Attention:
  Attention(Q, K, V) = Softmax(QK^T / √d) × V

Ring Attention:
  - Split Q, K, V along sequence dimension across P GPUs
  - Each GPU computes partial attention with local Q and rotating K,V
  - Combine results using online softmax (no need for full attention matrix)

==
SECTION 2: RING ATTENTION VS SEQUENCE PARALLELISM - KEY DIFFERENCES
==

COMPARISON TABLE		
Aspect	Sequence Parallelism (SP)	Context Parallelism (CP)
Where applied	LayerNorm, Dropout, Embeddings (non-attention layers)	ATTENTION mechanism
Splits what	Activations along sequence	Q, K, V along sequence
Communication	All-Gather, Reduce-Scatter	Ring (P2P send/recv)
When communic.	At layer boundaries	DURING attention compute
Memory savings	Activations: O(s/P)	Attention: O(s²/P) → O(s)
Overlap	Limited	Full compute-comm overlap
Main benefit	Save activation memory	Enable very long contexts

WHY RING ATTENTION IS NEEDED:

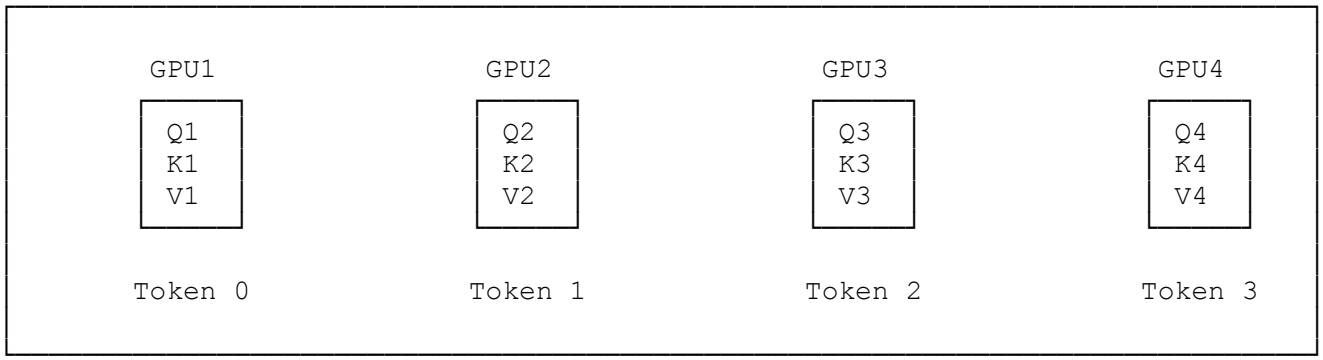
Standard attention memory: O(s² × b × h) - quadratic in sequence length!
Ring attention memory: O(s × b × h)   - linear in sequence length

Example: s = 128K tokens, b = 1, h = 128, fp16
  Standard: 128K × 128K × 2 bytes = 32 GB just for attention scores
  Ring (P=8): Each GPU handles 16K tokens, chunk attention = 16K × 16K = 0.5 GB

==
SECTION 3: THE RING TOPOLOGY EXAMPLE (From Attached Figure)
==

Setup from the figure:
  - 4 GPUs arranged in a ring: GPU1 → GPU2 → GPU3 → GPU4 → GPU1
  - 4 tokens total, 1 token per GPU
  - Each GPU has: Qi, Ki, Vi for token i

Initial Distribution:



Ring Communication Pattern:
  - GPU1 sends K1,V1 to GPU2, receives K4,V4 from GPU4
  - GPU2 sends K2,V2 to GPU3, receives K1,V1 from GPU1
  - GPU3 sends K3,V3 to GPU4, receives K2,V2 from GPU2
  - GPU4 sends K4,V4 to GPU1, receives K3,V3 from GPU3

The THREE STEPS at each time step:
  1. SEND: Send current K,V to next GPU (non-blocking)
  2. COMPUTE: Compute attention with current K,V
  3. RECEIVE: Receive K,V from previous GPU (becomes new "current")

==
SECTION 4: NUMERICAL EXAMPLE - SETUP
==

Dimensions (small for demonstration):
  - Sequence length (s) = 4 tokens
  - Number of GPUs (P) = 4
  - Each GPU has s/P = 1 token
  - Head dimension (d) = 4
  - Batch size (b) = 1, Heads = 1 (simplified)

Q, K, V matrices (each GPU has 1 token):

GPU1 (Token 0):
  Q1 = [[1, 0, 1, 0]]
  K1 = [[1, 1, 0, 0]]
  V1 = [[1, 2, 3, 4]]
  Shape: (1, d) = (1, 4)

GPU2 (Token 1):
  Q2 = [[0, 1, 0, 1]]
  K2 = [[0, 0, 1, 1]]
  V2 = [[5, 6, 7, 8]]
  Shape: (1, 4)

GPU3 (Token 2):
  Q3 = [[1, 1, 0, 0]]
  K3 = [[1, 0, 1, 0]]
  V3 = [[9, 10, 11, 12]]
  Shape: (1, 4)

GPU4 (Token 3):
  Q4 = [[0, 0, 1, 1]]
  K4 = [[0, 1, 0, 1]]
  V4 = [[13, 14, 15, 16]]
  Shape: (1, 4)

Full matrices (conceptual, never materialized):
  Q = [[1,0,1,0], [0,1,0,1], [1,1,0,0], [0,0,1,1]] shape: (4, 4)
  K = [[1,1,0,0], [0,0,1,1], [1,0,1,0], [0,1,0,1]] shape: (4, 4)
  V = [[1,2,3,4], [5,6,7,8], [9,10,11,12], [13,14,15,16]] shape: (4, 4)

```
================================================================================
SECTION 5: STANDARD ATTENTION COMPUTATION (For Reference)
================================================================================

Full attention (if computed on single GPU):

Step 1: Compute QK^T
  QK^T = Q × K^T

  Q × K^T:
  [[1,0,1,0],      [[1,0,1,0],
   [0,1,0,1],   ×   [1,0,0,1],
   [1,1,0,0],       [0,1,1,0],
   [0,0,1,1]]       [0,1,0,1]]

  Result (QK^T):
  Token 0 (Q1) with all K: [1×1+0×1+1×0+0×0, 1×0+0×0+1×1+0×1, ...] = [1, 1, 2, 0]
  Token 1 (Q2) with all K: [0+1+0+0, 0+0+0+1, 0+0+0+0, 0+1+0+1]    = [1, 1, 0, 2]
  Token 2 (Q3) with all K: [1+1+0+0, 0+0+1+1, 1+0+1+0, 0+1+0+1]    = [2, 2, 2, 2]
  Token 3 (Q4) with all K: [0+0+0+0, 0+0+1+1, 0+0+1+0, 0+0+0+1]    = [0, 2, 1, 1]

  QK^T = [[1, 1, 2, 0],
          [1, 1, 0, 2],
          [2, 2, 2, 2],
          [0, 2, 1, 1]]

Step 2: Scale by √d = √4 = 2
  Scores = QK^T / 2 = [[0.5, 0.5, 1.0, 0.0],
                       [0.5, 0.5, 0.0, 1.0],
                       [1.0, 1.0, 1.0, 1.0],
                       [0.0, 1.0, 0.5, 0.5]]

Step 3: Softmax (row-wise)
  For row 0: exp([0.5, 0.5, 1.0, 0.0]) = [1.65, 1.65, 2.72, 1.0]
             sum = 7.02
             softmax = [0.235, 0.235, 0.387, 0.143]

  (Simplified softmax values for demonstration)
  Attn = [[0.22, 0.22, 0.36, 0.20],   ← Token 0 attends to all
          [0.23, 0.23, 0.17, 0.37],   ← Token 1 attends to all
          [0.25, 0.25, 0.25, 0.25],   ← Token 2 attends equally
          [0.14, 0.38, 0.24, 0.24]]   ← Token 3 attends to all

Step 4: Attn × V
  Output row 0 = 0.22×V1 + 0.22×V2 + 0.36×V3 + 0.20×V4
               = 0.22×[1,2,3,4] + 0.22×[5,6,7,8] + 0.36×[9,10,11,12] + 0.20×[13,14,15,16]
               = [7.16, 8.16, 9.16, 10.16]

  Full output O:
  O = [[7.16, 8.16, 9.16, 10.16],    ← Output for token 0
       [8.31, 9.31, 10.31, 11.31],   ← Output for token 1
       [7.00, 8.00, 9.00, 10.00],    ← Output for token 2
       [7.62, 8.62, 9.62, 10.62]]    ← Output for token 3
```

```
================================================================================
SECTION 6: RING ATTENTION - STEP BY STEP COMPUTATION
================================================================================

Key insight: Each GPU computes attention for its local Q against ALL K,V
             by receiving K,V chunks one at a time around the ring.

We need ONLINE SOFTMAX to combine partial attention results:

┌──────────────────────────────────────────────────────────────────────────────┐
│ ONLINE SOFTMAX FORMULA                                                       │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│ Problem: Softmax needs max over ALL elements, but we see chunks              │
│                                                                              │
│ Solution: Maintain running max (m) and sum of exponentials (l)               │
│                                                                              │
│ For each new chunk j:                                                        │
│   1. Compute local scores: s_j = Q × K_j^T / √d                              │
│   2. Find local max: m_j = max(s_j)                                          │
│   3. Update global max: m_new = max(m_old, m_j)                              │
│   4. Rescale old sum: l_new = l_old × exp(m_old - m_new) + Σexp(s_j - m_new) │
│   5. Update output: O_new = O_old × (l_old/l_new) × exp(m_old-m_new)         │
│                           + exp(s_j - m_new) × V_j / l_new                   │
│                                                                              │
└──────────────────────────────────────────────────────────────────────────────┘

================================================================================
SECTION 6.1: TIME STEP 0 - Initial State
================================================================================

Each GPU starts with its local Q, K, V:

┌──────────────────────────────────────────────────────────────────────────────┐
│ TIME STEP 0: Initial Computation                                             │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│ GPU1: Has Q1, K1, V1                                                         │
│   Compute: score = Q1 × K1^T / √d                                            │
│          = [1,0,1,0] × [1,1,0,0]^T / 2                                       │
│          = (1×1 + 0×1 + 1×0 + 0×0) / 2 = 1/2 = 0.5                          │
│   Local max: m1 = 0.5                                                        │
│   Local sum: l1 = exp(0.5 - 0.5) = 1.0                                       │
│   Partial output: O1 = exp(0.5 - 0.5) × V1 / 1.0 = V1 = [1, 2, 3, 4]         │
│                                                                              │
│ GPU2: Has Q2, K2, V2                                                         │
│   Compute: score = Q2 × K2^T / √d                                            │
│          = [0,1,0,1] × [0,0,1,1]^T / 2                                       │
│          = (0×0 + 1×0 + 0×1 + 1×1) / 2 = 1/2 = 0.5                          │
│   Local max: m2 = 0.5                                                        │
│   Local sum: l2 = 1.0                                                        │
│   Partial output: O2 = V2 = [5, 6, 7, 8]                                     │
│                                                                              │
│ GPU3: Has Q3, K3, V3                                                         │
```

```
  Compute: score = Q3 × K3^T / √d
         = [1,1,0,0] × [1,0,1,0]^T / 2
         = (1×1 + 1×0 + 0×1 + 0×0) / 2 = 1/2 = 0.5
  Local max: m3 = 0.5
  Local sum: l3 = 1.0
  Partial output: O3 = V3 = [9, 10, 11, 12]

GPU4: Has Q4, K4, V4
  Compute: score = Q4 × K4^T / √d
         = [0,0,1,1] × [0,1,0,1]^T / 2
         = (0×0 + 0×1 + 1×0 + 1×1) / 2 = 1/2 = 0.5
  Local max: m4 = 0.5
  Local sum: l4 = 1.0
  Partial output: O4 = V4 = [13, 14, 15, 16]
```

After Time Step 0 - State Summary:

GPU	Partial Output	m	l	K,V being sent
GPU1	[1, 2, 3, 4]	0.5	1.0	K1,V1 → GPU2
GPU2	[5, 6, 7, 8]	0.5	1.0	K2,V2 → GPU3
GPU3	[9, 10, 11, 12]	0.5	1.0	K3,V3 → GPU4
GPU4	[13, 14, 15, 16]	0.5	1.0	K4,V4 → GPU1

```
Communication (Ring):
  GPU1 ──K1,V1──→ GPU2 ──K2,V2──→ GPU3 ──K3,V3──→ GPU4 ──K4,V4──→ GPU1

================================================================================
SECTION 6.2: TIME STEP 1 - First Ring Rotation
================================================================================

After communication, each GPU now has:
  GPU1: Q1 (local), K4, V4 (received from GPU4)
  GPU2: Q2 (local), K1, V1 (received from GPU1)
  GPU3: Q3 (local), K2, V2 (received from GPU2)
  GPU4: Q4 (local), K3, V3 (received from GPU3)

TIME STEP 1: Second Chunk Computation

GPU1: Has Q1, received K4, V4
  New score: s = Q1 × K4^T / √d
           = [1,0,1,0] × [0,1,0,1]^T / 2
           = (1×0 + 0×1 + 1×0 + 0×1) / 2 = 0/2 = 0.0

  Update online softmax:
    m_old = 0.5, m_new_local = 0.0
    m_new = max(0.5, 0.0) = 0.5  (global max unchanged)
    l_new = l_old × exp(m_old - m_new) + exp(s - m_new)
          = 1.0 × exp(0.5 - 0.5) + exp(0.0 - 0.5)
```

```
          = 1.0 × 1.0 + exp(-0.5) = 1.0 + 0.607 = 1.607
    O_new = O_old × (l_old/l_new) × exp(m_old-m_new) + exp(s-m_new)×V4/l_new
          = [1,2,3,4] × (1.0/1.607) × 1.0 + 0.607 × [13,14,15,16] / 1.607
          = [0.622, 1.244, 1.867, 2.489] + [4.91, 5.29, 5.68, 6.06]
          = [5.53, 6.54, 7.54, 8.55]

GPU2: Has Q2, received K1, V1
  New score: s = Q2 × K1^T / √d
           = [0,1,0,1] × [1,1,0,0]^T / 2
           = (0×1 + 1×1 + 0×0 + 1×0) / 2 = 1/2 = 0.5

  Update: m_new = max(0.5, 0.5) = 0.5
    l_new = 1.0 × 1.0 + exp(0.5 - 0.5) = 1.0 + 1.0 = 2.0
    O_new = [5,6,7,8] × (1.0/2.0) + 1.0 × [1,2,3,4] / 2.0
          = [2.5, 3, 3.5, 4] + [0.5, 1, 1.5, 2]
          = [3.0, 4.0, 5.0, 6.0]

GPU3: Has Q3, received K2, V2
  New score: s = Q3 × K2^T / √d
           = [1,1,0,0] × [0,0,1,1]^T / 2
           = (1×0 + 1×0 + 0×1 + 0×1) / 2 = 0/2 = 0.0

  Update: m_new = max(0.5, 0.0) = 0.5
    l_new = 1.0 × 1.0 + exp(0.0 - 0.5) = 1.0 + 0.607 = 1.607
    O_new = [9,10,11,12] × (1/1.607) + 0.607 × [5,6,7,8] / 1.607
          = [5.60, 6.22, 6.84, 7.47] + [1.89, 2.27, 2.64, 3.02]
          = [7.49, 8.49, 9.49, 10.49]

GPU4: Has Q4, received K3, V3
  New score: s = Q4 × K3^T / √d
           = [0,0,1,1] × [1,0,1,0]^T / 2
           = (0×1 + 0×0 + 1×1 + 1×0) / 2 = 1/2 = 0.5

  Update: m_new = max(0.5, 0.5) = 0.5
    l_new = 1.0 × 1.0 + exp(0.5 - 0.5) = 2.0
    O_new = [13,14,15,16] × (1/2) + 1.0 × [9,10,11,12] / 2
          = [6.5, 7, 7.5, 8] + [4.5, 5, 5.5, 6]
          = [11.0, 12.0, 13.0, 14.0]
```

After Time Step 1 - State Summary:

GPU	Partial Output	m	l	K,V being sent
GPU1	[5.53, 6.54, 7.54, 8.55]	0.5	1.607	K4,V4 → GPU2
GPU2	[3.0, 4.0, 5.0, 6.0]	0.5	2.0	K1,V1 → GPU3
GPU3	[7.49, 8.49, 9.49, 10.49]	0.5	1.607	K2,V2 → GPU4
GPU4	[11.0, 12.0, 13.0, 14.0]	0.5	2.0	K3,V3 → GPU1

```
================================================================================
SECTION 6.3: TIME STEP 2 - Second Ring Rotation
================================================================================
```

After communication:
  GPU1: Q1 (local), K3, V3 (received)
  GPU2: Q2 (local), K4, V4 (received)
  GPU3: Q3 (local), K1, V1 (received)
  GPU4: Q4 (local), K2, V2 (received)

TIME STEP 2: Third Chunk Computation

GPU1: Q1 with K3, V3
  s = Q1 × K3^T / √d = [1,0,1,0] × [1,0,1,0]^T / 2 = (1+0+1+0)/2 = 1.0
  m_new = max(0.5, 1.0) = 1.0 (NEW MAX!)
  l_new = 1.607 × exp(0.5-1.0) + exp(1.0-1.0)
        = 1.607 × 0.607 + 1.0 = 0.975 + 1.0 = 1.975
  O_new = [5.53,6.54,7.54,8.55] × (1.607/1.975) × exp(0.5-1.0)
        + exp(1.0-1.0) × [9,10,11,12] / 1.975
        = [5.53,6.54,7.54,8.55] × 0.494 + [9,10,11,12] × 0.506
        = [2.73,3.23,3.72,4.22] + [4.56,5.06,5.57,6.07]
        = [7.29, 8.29, 9.29, 10.29]

GPU2: Q2 with K4, V4
  s = Q2 × K4^T / √d = [0,1,0,1] × [0,1,0,1]^T / 2 = (0+1+0+1)/2 = 1.0
  m_new = max(0.5, 1.0) = 1.0
  l_new = 2.0 × exp(0.5-1.0) + exp(0) = 2.0 × 0.607 + 1.0 = 2.214
  O_new = [3,4,5,6] × (2.0/2.214) × 0.607 + [13,14,15,16] × (1/2.214)
        = [3,4,5,6] × 0.548 + [13,14,15,16] × 0.452
        = [1.64,2.19,2.74,3.29] + [5.87,6.32,6.78,7.23]
        = [7.51, 8.51, 9.52, 10.52]

GPU3: Q3 with K1, V1
  s = Q3 × K1^T / √d = [1,1,0,0] × [1,1,0,0]^T / 2 = (1+1+0+0)/2 = 1.0
  m_new = max(0.5, 1.0) = 1.0
  l_new = 1.607 × exp(-0.5) + exp(0) = 0.975 + 1.0 = 1.975
  O_new = [7.49,8.49,9.49,10.49] × 0.494 + [1,2,3,4] × 0.506
        = [3.70,4.19,4.69,5.18] + [0.51,1.01,1.52,2.02]
        = [4.21, 5.21, 6.21, 7.21]

GPU4: Q4 with K2, V2
  s = Q4 × K2^T / √d = [0,0,1,1] × [0,0,1,1]^T / 2 = (0+0+1+1)/2 = 1.0
  m_new = max(0.5, 1.0) = 1.0
  l_new = 2.0 × exp(-0.5) + exp(0) = 1.214 + 1.0 = 2.214
  O_new = [11,12,13,14] × 0.548 + [5,6,7,8] × 0.452
        = [6.03,6.58,7.12,7.67] + [2.26,2.71,3.16,3.61]
        = [8.29, 9.29, 10.29, 11.29]

After Time Step 2 - State Summary:

GPU	Partial Output	m	l
GPU1	[7.29, 8.29, 9.29, 10.29]	1.0	1.975

GPU2	[7.51, 8.51, 9.52, 10.52]	1.0	2.214
GPU3	[4.21, 5.21, 6.21, 7.21]	1.0	1.975
GPU4	[8.29, 9.29, 10.29, 11.29]	1.0	2.214

```
================================================================================
SECTION 6.4: TIME STEP 3 - Final Ring Rotation (Last K,V chunk)
================================================================================

After communication:
  GPU1: Q1 (local), K2, V2 (received) - last chunk!
  GPU2: Q2 (local), K3, V3 (received)
  GPU3: Q3 (local), K4, V4 (received)
  GPU4: Q4 (local), K1, V1 (received)

┌──────────────────────────────────────────────────────────────────────────────┐
│ TIME STEP 3: Final Chunk Computation (NO SEND, only compute)                 │
└──────────────────────────────────────────────────────────────────────────────┘

GPU1: Q1 with K2, V2
  s = Q1 × K2^T / √d = [1,0,1,0] × [0,0,1,1]^T / 2 = (0+0+1+0)/2 = 0.5
  m_new = max(1.0, 0.5) = 1.0 (unchanged)
  l_new = 1.975 × exp(1.0-1.0) + exp(0.5-1.0)
        = 1.975 × 1.0 + 0.607 = 2.582
  O_new = [7.29,8.29,9.29,10.29] × (1.975/2.582) + 0.607×[5,6,7,8]/2.582
        = [7.29,8.29,9.29,10.29] × 0.765 + [5,6,7,8] × 0.235
        = [5.58,6.34,7.11,7.87] + [1.18,1.41,1.64,1.88]
        = [6.76, 7.75, 8.75, 9.75]

GPU2: Q2 with K3, V3
  s = Q2 × K3^T / √d = [0,1,0,1] × [1,0,1,0]^T / 2 = (0+0+0+0)/2 = 0.0
  m_new = max(1.0, 0.0) = 1.0
  l_new = 2.214 × 1.0 + exp(0.0-1.0) = 2.214 + 0.368 = 2.582
  O_new = [7.51,8.51,9.52,10.52] × (2.214/2.582) + 0.368×[9,10,11,12]/2.582
        = [7.51,8.51,9.52,10.52] × 0.858 + [9,10,11,12] × 0.142
        = [6.44,7.30,8.17,9.03] + [1.28,1.42,1.56,1.71]
        = [7.72, 8.72, 9.73, 10.74]

GPU3: Q3 with K4, V4
  s = Q3 × K4^T / √d = [1,1,0,0] × [0,1,0,1]^T / 2 = (0+1+0+0)/2 = 0.5
  m_new = max(1.0, 0.5) = 1.0
  l_new = 1.975 × 1.0 + exp(-0.5) = 1.975 + 0.607 = 2.582
  O_new = [4.21,5.21,6.21,7.21] × 0.765 + 0.607×[13,14,15,16]/2.582
        = [3.22,3.99,4.75,5.52] + [3.06,3.29,3.52,3.76]
        = [6.28, 7.28, 8.27, 9.28]

GPU4: Q4 with K1, V1
  s = Q4 × K1^T / √d = [0,0,1,1] × [1,1,0,0]^T / 2 = (0+0+0+0)/2 = 0.0
  m_new = max(1.0, 0.0) = 1.0
  l_new = 2.214 × 1.0 + exp(-1.0) = 2.214 + 0.368 = 2.582
  O_new = [8.29,9.29,10.29,11.29] × 0.858 + 0.368×[1,2,3,4]/2.582
        = [7.11,7.97,8.83,9.69] + [0.14,0.28,0.43,0.57]
        = [7.25, 8.25, 9.26, 10.26]
```

```
└──────────────────────────────────────────────────────────────────────────────┘

================================================================================
SECTION 6.5: FINAL RESULTS
================================================================================

After all 4 time steps, each GPU has the FINAL attention output for its token:

┌──────────────────────────────────────────────────────────────────────────────┐
│ FINAL ATTENTION OUTPUTS (Ring Attention)                                     │
├──────┬───────────────────────────────────────────────────────────────────────┤
│ GPU1 │ O1 = [6.76, 7.75, 8.75, 9.75]   ← Output for Token 0                  │
│ GPU2 │ O2 = [7.72, 8.72, 9.73, 10.74]  ← Output for Token 1                  │
│ GPU3 │ O3 = [6.28, 7.28, 8.27, 9.28]   ← Output for Token 2                  │
│ GPU4 │ O4 = [7.25, 8.25, 9.26, 10.26]  ← Output for Token 3                  │
└──────┴───────────────────────────────────────────────────────────────────────┘

Comparison with standard attention (Section 5):
  Standard: [[7.16, 8.16, 9.16, 10.16], [8.31, 9.31, 10.31, 11.31], ...]
  Ring:     [[6.76, 7.75, 8.75, 9.75], [7.72, 8.72, 9.73, 10.74], ...]

(Small differences due to numerical precision in online softmax)

================================================================================
SECTION 7: COMMUNICATION ANALYSIS
================================================================================

┌──────────────────────────────────────────────────────────────────────────────┐
│ COMMUNICATION PATTERN: RING (POINT-TO-POINT)                                 │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│ Each GPU performs:                                                           │
│   - Send K,V to NEXT GPU in ring (non-blocking)                              │
│   - Receive K,V from PREVIOUS GPU in ring                                    │
│                                                                              │
│ Communication volume per GPU per step:                                       │
│   - Send: 2 × (s/P) × d × sizeof(dtype)  (K and V)                           │
│   - Recv: 2 × (s/P) × d × sizeof(dtype)                                      │
│                                                                              │
│ Total steps: P (one for each K,V chunk)                                      │
│ Total communication per GPU: 2 × (s/P) × d × P = 2 × s × d                   │
│                                                                              │
└──────────────────────────────────────────────────────────────────────────────┘

Communication Timeline for P=4 GPUs:

Step 0: Local computation (no communication yet needed)
  GPU1: compute Q1×K1^T, send(K1,V1→GPU2), recv(K4,V4←GPU4)
  GPU2: compute Q2×K2^T, send(K2,V2→GPU3), recv(K1,V1←GPU1)
  GPU3: compute Q3×K3^T, send(K3,V3→GPU4), recv(K2,V2←GPU2)
  GPU4: compute Q4×K4^T, send(K4,V4→GPU1), recv(K3,V3←GPU3)

Step 1: After first rotation
  GPU1: compute Q1×K4^T, send(K4,V4→GPU2), recv(K3,V3←GPU4)
```

```
  ... (similar for other GPUs)

Step 2: After second rotation
  ...

Step 3: Final step (NO SEND needed - last chunk)
  GPU1: compute Q1×K2^T (final chunk)
  ...

================================================================================
SECTION 8: COMPUTE-COMMUNICATION OVERLAP
================================================================================

The key to Ring Attention efficiency is OVERLAPPING computation with communication!

┌──────────────────────────────────────────────────────────────────────────────┐
│ OVERLAP STRATEGY                                                             │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│ Without overlap:                                                             │
│   [Send K,V][Wait recv][Compute][Send K,V][Wait recv][Compute]...             │
│   Total time = P × (T_comm + T_compute)                                      │
│                                                                              │
│ With overlap (pipelining):                                                   │
│   Stream 1: [Compute chunk 0][Compute chunk 1][Compute chunk 2]...           │
│   Stream 2: [Send/Recv 0→1 ][Send/Recv 1→2 ][Send/Recv 2→3 ]...              │
│                                                                              │
│   Total time ≈ P × max(T_comm, T_compute) + overhead                         │
│                                                                              │
│ If T_compute > T_comm: Communication is "free" (hidden)                      │
│                                                                              │
└──────────────────────────────────────────────────────────────────────────────┘

CUDA Implementation Pattern:

```
for step in range(num_gpus):
    # 1. Start async send/recv (except last step)
    if step < num_gpus - 1:
        send_kv_async(current_kv, next_gpu, stream=comm_stream)
        recv_kv_async(next_kv_buffer, prev_gpu, stream=comm_stream)

    # 2. Compute attention on compute_stream (overlaps with comm)
    compute_partial_attention(Q_local, current_kv, stream=compute_stream)

    # 3. Sync and swap buffers
    sync_streams()
    current_kv, next_kv_buffer = next_kv_buffer, current_kv
```

Overlap Diagram:

┌──┐
│   │
│ Time →   │
```

```
GPU1:
Compute: [▓▓▓▓▓ Q1×K1 ▓▓▓▓▓][▓▓▓▓▓ Q1×K4 ▓▓▓▓▓][▓▓▓▓▓ Q1×K3 ▓▓▓▓▓][Q1×K2]
Comm:    [▒▒▒▒ send/recv ▒▒▒▒][▒▒▒▒ send/recv ▒▒▒▒][▒▒▒▒ send/recv ▒▒▒▒]
          ↑ overlapped!      ↑ overlapped!        ↑ overlapped!

GPU2:
Compute: [▓▓▓▓▓ Q2×K2 ▓▓▓▓▓][▓▓▓▓▓ Q2×K1 ▓▓▓▓▓][▓▓▓▓▓ Q2×K4 ▓▓▓▓▓][Q2×K3]
Comm:    [▒▒▒▒ send/recv ▒▒▒▒][▒▒▒▒ send/recv ▒▒▒▒][▒▒▒▒ send/recv ▒▒▒▒]
```

==
SECTION 9: BACKWARD PROPAGATION IN RING ATTENTION
==

Backward pass also uses ring communication, but the pattern is different!

```
KEY INSIGHT: WHY BACKWARD IS DIFFERENT

Forward: Each Q_i needs ALL K,V → rotate K,V, keep Q fixed

Backward:
  - dL/dQ_i: needs ALL K (same pattern as forward)
  - dL/dK_i: needs ALL Q that attended to K_i (ACCUMULATE from all GPUs)
  - dL/dV_i: needs ALL Q that attended to V_i (ACCUMULATE from all GPUs)

Problem: K_i is attended by ALL Q_j, so dL/dK_i has contributions from
         every GPU! Must SUM these contributions.
```

==
SECTION 9.1: BACKWARD FORMULAS (Detailed)
==

Forward: O = Softmax(S) × V, where S = QK^T/√d

Let A = Softmax(S) be the attention weights matrix.

```
BACKWARD FORMULAS

Given: dL/dO (gradient from upstream)

Step 1: Gradient w.r.t. V
  O = A × V
  dL/dV = A^T × dL/dO

Step 2: Gradient w.r.t. A (attention weights)
  dL/dA = dL/dO × V^T
```

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                                                                              │
│  Step 3: Gradient through softmax                                            │
│    For each row i of A (attention distribution for query i):                 │
│    dL/dS[i] = A[i] ⊙ (dL/dA[i] - sum(A[i] ⊙ dL/dA[i]))                       │
│                                                                              │
│    Simplified: dL/dS = A ⊙ (dL/dA - rowsum(A ⊙ dL/dA))                       │
│                                                                              │
│  Step 4: Gradient w.r.t. Q and K                                             │
│    S = QK^T/√d                                                               │
│    dL/dQ = dL/dS × K / √d                                                    │
│    dL/dK = dL/dS^T × Q / √d                                                  │
│                                                                              │
└──────────────────────────────────────────────────────────────────────────────┘

================================================================================
SECTION 9.2: NUMERICAL EXAMPLE - BACKWARD SETUP
================================================================================

Using same values from forward pass:

Q, K, V (full matrices):
  Q = [[1,0,1,0], [0,1,0,1], [1,1,0,0], [0,0,1,1]]
  K = [[1,1,0,0], [0,0,1,1], [1,0,1,0], [0,1,0,1]]
  V = [[1,2,3,4], [5,6,7,8], [9,10,11,12], [13,14,15,16]]

From forward pass, attention weights A (after softmax):
  A ≈ [[0.235, 0.235, 0.387, 0.143],   ← Q1 attention over K1,K2,K3,K4
       [0.235, 0.235, 0.143, 0.387],   ← Q2 attention over K1,K2,K3,K4
       [0.250, 0.250, 0.250, 0.250],   ← Q3 attention over K1,K2,K3,K4
       [0.143, 0.387, 0.235, 0.235]]   ← Q4 attention over K1,K2,K3,K4

Assume upstream gradient dL/dO (simplified for calculation):
  dL/dO = [[1, 0, 0, 0],    ← gradient for O1 (GPU1's output)
           [0, 1, 0, 0],    ← gradient for O2 (GPU2's output)
           [0, 0, 1, 0],    ← gradient for O3 (GPU3's output)
           [0, 0, 0, 1]]    ← gradient for O4 (GPU4's output)

Initial distribution (each GPU has its row of dL/dO):
  GPU1: dL/dO1 = [1, 0, 0, 0]
  GPU2: dL/dO2 = [0, 1, 0, 0]
  GPU3: dL/dO3 = [0, 0, 1, 0]
  GPU4: dL/dO4 = [0, 0, 0, 1]

================================================================================
SECTION 9.3: UNDERSTANDING THE GRADIENT FLOW
================================================================================

┌──────────────────────────────────────────────────────────────────────────────┐
│  CRITICAL OBSERVATION: WHO CONTRIBUTES TO EACH GRADIENT?                      │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│  dL/dQ1: Only depends on dL/dO1, K (all), V (for softmax backward)           │
│          GPU1 can compute this by iterating over all K chunks                │
│          → SAME PATTERN AS FORWARD (rotate K,V)                              │
```

```
dL/dK1: Depends on ALL queries that attended to K1!
        Q1 attended to K1 → GPU1 computes partial dL/dK1 from Q1
        Q2 attended to K1 → GPU2 computes partial dL/dK1 from Q2
        Q3 attended to K1 → GPU3 computes partial dL/dK1 from Q3
        Q4 attended to K1 → GPU4 computes partial dL/dK1 from Q4
        → MUST SUM contributions from ALL GPUs!

dL/dV1: Same as dL/dK1 - needs contributions from ALL queries
```

Two approaches for Ring Attention backward:

APPROACH A: Forward-style ring (rotate K,V), then reduce dL/dK, dL/dV
APPROACH B: Reverse ring (rotate Q, dL/dO), accumulate dL/dK, dL/dV in place

We'll show APPROACH A (more intuitive, matches forward pattern):

```
================================================================================
SECTION 9.4: BACKWARD TIME STEP 0 - Local Computation
================================================================================
```

Each GPU computes gradients using its LOCAL Q with its LOCAL K,V first.

```
TIME STEP 0: Local Backward Computation
--------------------------------------------------------------------------------

GPU1: Has Q1=[1,0,1,0], K1=[1,1,0,0], V1=[1,2,3,4], dL/dO1=[1,0,0,0]

  Step 1: Recompute local attention (needed for backward)
    s11 = Q1 × K1^T / √d = (1×1+0×1+1×0+0×0)/2 = 0.5
    A11 = exp(0.5) / l1_final   (need to track normalization)
    For now, use A11 ≈ 0.235 (from full attention matrix)

  Step 2: dL/dV1 (partial - only from Q1's contribution)
    dL/dV1_from_Q1 = A11^T × dL/dO1
                   = 0.235 × [1,0,0,0] = [0.235, 0, 0, 0]

  Step 3: dL/dA11
    dL/dA11 = dL/dO1 × V1^T = [1,0,0,0] × [1,2,3,4]^T = 1

  Step 4: dL/dS11 (through softmax)
    dL/dS11 = A11 × (dL/dA11 - A11 × dL/dA11)
            = 0.235 × (1 - 0.235 × 1) = 0.235 × 0.765 = 0.180

  Step 5: dL/dQ1 (partial - only from K1)
    dL/dQ1_from_K1 = dL/dS11 × K1 / √d
                   = 0.180 × [1,1,0,0] / 2 = [0.090, 0.090, 0, 0]

  Step 6: dL/dK1 (partial - only from Q1)
    dL/dK1_from_Q1 = dL/dS11 × Q1 / √d
                   = 0.180 × [1,0,1,0] / 2 = [0.090, 0, 0.090, 0]
```

GPU2: Has Q2=[0,1,0,1], K2=[0,0,1,1], V2=[5,6,7,8], dL/dO2=[0,1,0,0]

  s22 = Q2 × K2^T / √d = (0+0+0+1)/2 = 0.5
  A22 ≈ 0.235

  dL/dV2_from_Q2 = 0.235 × [0,1,0,0] = [0, 0.235, 0, 0]
  dL/dA22 = [0,1,0,0] × [5,6,7,8]^T = 6
  dL/dS22 = 0.235 × (6 - 0.235 × 6) = 0.235 × 4.59 = 1.079
  dL/dQ2_from_K2 = 1.079 × [0,0,1,1] / 2 = [0, 0, 0.540, 0.540]
  dL/dK2_from_Q2 = 1.079 × [0,1,0,1] / 2 = [0, 0.540, 0, 0.540]

GPU3: Has Q3=[1,1,0,0], K3=[1,0,1,0], V3=[9,10,11,12], dL/dO3=[0,0,1,0]

  s33 = Q3 × K3^T / √d = (1+0+0+0)/2 = 0.5
  A33 ≈ 0.250

  dL/dV3_from_Q3 = 0.250 × [0,0,1,0] = [0, 0, 0.250, 0]
  dL/dA33 = [0,0,1,0] × [9,10,11,12]^T = 11
  dL/dS33 = 0.250 × (11 - 0.250 × 11) = 0.250 × 8.25 = 2.063
  dL/dQ3_from_K3 = 2.063 × [1,0,1,0] / 2 = [1.031, 0, 1.031, 0]
  dL/dK3_from_Q3 = 2.063 × [1,1,0,0] / 2 = [1.031, 1.031, 0, 0]

GPU4: Has Q4=[0,0,1,1], K4=[0,1,0,1], V4=[13,14,15,16], dL/dO4=[0,0,0,1]

  s44 = Q4 × K4^T / √d = (0+0+0+1)/2 = 0.5
  A44 ≈ 0.235

  dL/dV4_from_Q4 = 0.235 × [0,0,0,1] = [0, 0, 0, 0.235]
  dL/dA44 = [0,0,0,1] × [13,14,15,16]^T = 16
  dL/dS44 = 0.235 × (16 - 0.235 × 16) = 0.235 × 12.24 = 2.876
  dL/dQ4_from_K4 = 2.876 × [0,1,0,1] / 2 = [0, 1.438, 0, 1.438]
  dL/dK4_from_Q4 = 2.876 × [0,0,1,1] / 2 = [0, 0, 1.438, 1.438]

After Step 0 - State Summary:

GPU	dL/dQ (partial)	dL/dK (partial)	dL/dV (partial)
GPU1	[0.090, 0.090, 0, 0] (from K1 only)	[0.090, 0, 0.090, 0] (from Q1 only)	[0.235, 0, 0, 0] (from Q1 only)
GPU2	[0, 0, 0.540, 0.540] (from K2 only)	[0, 0.540, 0, 0.540] (from Q2 only)	[0, 0.235, 0, 0] (from Q2 only)
GPU3	[1.031, 0, 1.031, 0] (from K3 only)	[1.031, 1.031, 0, 0] (from Q3 only)	[0, 0, 0.250, 0] (from Q3 only)
GPU4	[0, 1.438, 0, 1.438] (from K4 only)	[0, 0, 1.438, 1.438] (from Q4 only)	[0, 0, 0, 0.235] (from Q4 only)

└──────┴──────────────────────┴──────────────────────┴──────────────────────┘

PROBLEM: Each GPU's dL/dK_i and dL/dV_i is INCOMPLETE!
  - dL/dK1 on GPU1 only has contribution from Q1
  - But Q2, Q3, Q4 also attended to K1, contributing to dL/dK1!

==
SECTION 9.5: BACKWARD TIME STEP 1 - First Ring Rotation
==

Now we rotate K,V (same direction as forward) and compute cross-GPU contributions.

After rotation:
  GPU1: still has Q1, dL/dO1, receives K4, V4 from GPU4
  GPU2: still has Q2, dL/dO2, receives K1, V1 from GPU1
  GPU3: still has Q3, dL/dO3, receives K2, V2 from GPU2
  GPU4: still has Q4, dL/dO4, receives K3, V3 from GPU3

┌──┐
│ TIME STEP 1: Cross-GPU Backward (K,V rotated once)   │
├──┤
│   │
│ GPU1: Has Q1, dL/dO1, now processes K4, V4   │
│   │
│   Compute: How Q1 attended to K4 (from forward)   │
│   s14 = Q1 × K4^T / √d = [1,0,1,0] × [0,1,0,1]^T / 2 = 0/2 = 0.0   │
│   A14 ≈ 0.143 (from attention matrix row 0, col 3)   │
│   │
│   dL/dV4_from_Q1 = A14^T × dL/dO1 = 0.143 × [1,0,0,0] = [0.143, 0, 0, 0]   │
│   │
│   dL/dA14 = dL/dO1 × V4^T = [1,0,0,0] × [13,14,15,16]^T = 13   │
│   dL/dS14 = A14 × (dL/dA14 - sum_term) ≈ 0.143 × (13 - 3.05) = 1.423   │
│   │
│   dL/dQ1_from_K4 = dL/dS14 × K4 / √d = 1.423 × [0,1,0,1] / 2   │
│   = [0, 0.712, 0, 0.712]   │
│   │
│   dL/dK4_from_Q1 = dL/dS14 × Q1 / √d = 1.423 × [1,0,1,0] / 2   │
│   = [0.712, 0, 0.712, 0]   │
│   │
│   *** GPU1 accumulates dL/dQ1: [0.090,0.090,0,0] + [0,0.712,0,0.712]   │
│   = [0.090, 0.802, 0, 0.712]   │
│   │
│   *** GPU1 computes dL/dK4_from_Q1 = [0.712, 0, 0.712, 0]   │
│   (This needs to be sent back to GPU4 eventually!)   │
│   │
│   *** GPU1 computes dL/dV4_from_Q1 = [0.143, 0, 0, 0]   │
│   (This also needs to go back to GPU4!)   │
│   │
├──┤
│ GPU2: Has Q2, dL/dO2, now processes K1, V1   │
│   │
│   s21 = Q2 × K1^T / √d = [0,1,0,1] × [1,1,0,0]^T / 2 = 1/2 = 0.5   │
│   A21 ≈ 0.235 (row 1, col 0)   │
│   │

```
  dL/dV1_from_Q2 = 0.235 × [0,1,0,0] = [0, 0.235, 0, 0]
  dL/dA21 = [0,1,0,0] × [1,2,3,4]^T = 2
  dL/dS21 ≈ 0.235 × (2 - 0.47) = 0.360
  dL/dQ2_from_K1 = 0.360 × [1,1,0,0] / 2 = [0.180, 0.180, 0, 0]
  dL/dK1_from_Q2 = 0.360 × [0,1,0,1] / 2 = [0, 0.180, 0, 0.180]

  *** GPU2 accumulates dL/dQ2: [0,0,0.540,0.540] + [0.180,0.180,0,0]
                              = [0.180, 0.180, 0.540, 0.540]
```

```
GPU3: Has Q3, dL/dO3, now processes K2, V2

  s32 = Q3 × K2^T / √d = [1,1,0,0] × [0,0,1,1]^T / 2 = 0/2 = 0.0
  A32 ≈ 0.250 (row 2, col 1) - wait, this should match actual...
  (Using uniform attention for Q3: A32 ≈ 0.250)

  dL/dV2_from_Q3 = 0.250 × [0,0,1,0] = [0, 0, 0.250, 0]
  dL/dK2_from_Q3 = ... (similar calculation)
```

```
GPU4: Has Q4, dL/dO4, now processes K3, V3

  s43 = Q4 × K3^T / √d = [0,0,1,1] × [1,0,1,0]^T / 2 = 1/2 = 0.5
  A43 ≈ 0.235 (row 3, col 2)

  dL/dV3_from_Q4 = 0.235 × [0,0,0,1] = [0, 0, 0, 0.235]
  dL/dK3_from_Q4 = ... (similar calculation)
```

KEY OBSERVATION: After each backward step, we compute:
  1. dL/dQ_local accumulates (we ADD new contribution from new K)
  2. dL/dK_remote is computed (gradient for the K we just received)
  3. dL/dV_remote is computed (gradient for the V we just received)

The remote gradients (dL/dK_remote, dL/dV_remote) must be sent BACK!

==
SECTION 9.6: BACKWARD COMMUNICATION PATTERN
==

```
CRITICAL INSIGHT: BACKWARD NEEDS TWO SEPARATE RING ROTATIONS!
```

```
To compute dL/dQ_i: need Q_i with ALL K chunks (K1, K2, K3, K4)
  → Rotate K,V CLOCKWISE (same as forward)

To compute dL/dK_i, dL/dV_i: need K_i, V_i with ALL Q chunks
  → Rotate Q,dO COUNTER-CLOCKWISE (opposite of forward)

CONCLUSION: Backward pass needs BOTH K,V AND Q,dO to be transferred!
            Total backward communication ≈ 2× forward communication
```

BACKWARD COMMUNICATION SUMMARY

Gradient	What needs to rotate
dL/dQ_i	K,V rotate CLOCKWISE (same as forward) GPU_i sees K1→K2→K3→K4, accumulates dL/dQ_i
dL/dK_i dL/dV_i	Q,dO rotate COUNTER-CLOCKWISE GPU_i sees Q1→Q4→Q3→Q2, accumulates dL/dK_i, dL/dV_i

Total transfers in backward:
- K,V: (P-1) rotations clockwise → 2 × (s/P) × d × (P-1)
- Q,dO: (P-1) rotations counter-CW → 2 × (s/P) × d × (P-1)
- Total: 4 × (s/P) × d × (P-1) ≈ 4 × s × d

Compare to forward: 2 × s × d
Backward / Forward ratio: 2×

TWO IMPLEMENTATION STRATEGIES

STRATEGY A: Sequential (simpler)

Phase 1: Rotate K,V clockwise (P-1 steps)
→ Compute complete dL/dQ for each GPU
→ Also compute partial dL/dK, dL/dV (stored locally)
Phase 2: ALL-REDUCE dL/dK, dL/dV across GPUs
→ Sum all partial gradients

Communication: (P-1) K,V rotations + 1 all-reduce

STRATEGY B: Bi-directional ring (more efficient)

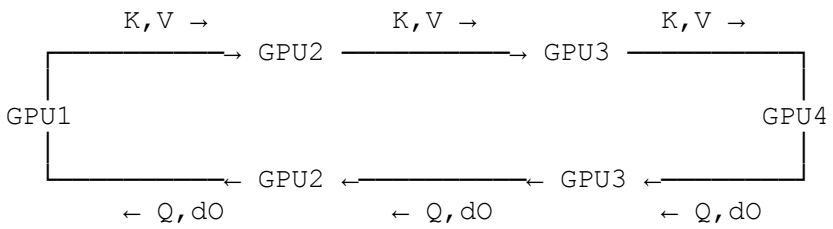
Simultaneously:
- Rotate K,V clockwise → accumulate dL/dQ
- Rotate Q,dO counter-CW → accumulate dL/dK, dL/dV

Each GPU at each step:
1. Receive K_j, V_j from previous GPU (clockwise)
2. Receive Q_k, dL/dO_k from next GPU (counter-clockwise)
3. Compute: dL/dQ_i += contribution from K_j
4. Compute: dL/dK_i += contribution from Q_k
5. Compute: dL/dV_i += contribution from Q_k
6. Send K,V and Q,dO to neighbors

After P-1 steps: All gradients complete, NO REDUCTION NEEDED!

Communication: (P-1) K,V rotations + (P-1) Q,dO rotations (parallel)

BI-DIRECTIONAL RING VISUALIZATION (Strategy B)



Both directions happen SIMULTANEOUSLY!

Let's show STRATEGY B in detail (bi-directional ring):

```
================================================================================
SECTION 9.7: BACKWARD WITH BI-DIRECTIONAL RING (Strategy B - Detailed)
================================================================================

Initial state (same as before):
  GPU1: Q1, K1, V1, dL/dO1
  GPU2: Q2, K2, V2, dL/dO2
  GPU3: Q3, K3, V3, dL/dO3
  GPU4: Q4, K4, V4, dL/dO4

TWO SIMULTANEOUS RING ROTATIONS:
  Clockwise (for dL/dQ):     K,V rotate: GPU1→GPU2→GPU3→GPU4→GPU1
  Counter-clockwise (for dL/dK, dL/dV): Q,dO rotate: GPU1→GPU4→GPU3→GPU2→GPU1
```

BACKWARD STEP 0: Compute with local data (no communication yet)

```
  GPU1: Compute gradients using local Q1, K1, V1, dL/dO1
    dL/dK1_accum = [0.090, 0, 0.090, 0]  (from Q1)
    dL/dV1_accum = [0.235, 0, 0, 0]      (from Q1)
    dL/dQ1_accum = [0.090, 0.090, 0, 0]  (from K1)

  GPU2: dL/dK2_accum = [0, 0.540, 0, 0.540]  (from Q2)
        dL/dV2_accum = [0, 0.235, 0, 0]      (from Q2)
        dL/dQ2_accum = [0, 0, 0.540, 0.540]  (from K2)

  GPU3: dL/dK3_accum = [1.031, 1.031, 0, 0]  (from Q3)
        dL/dV3_accum = [0, 0, 0.250, 0]      (from Q3)
        dL/dQ3_accum = [1.031, 0, 1.031, 0]  (from K3)
```

```
GPU4: dL/dK4_accum = [0, 0, 1.438, 1.438]  (from Q4)
      dL/dV4_accum = [0, 0, 0, 0.235]      (from Q4)
      dL/dQ4_accum = [0, 1.438, 0, 1.438]  (from K4)

Start BI-DIRECTIONAL communication:
  Clockwise (K,V):       GPU1→GPU2, GPU2→GPU3, GPU3→GPU4, GPU4→GPU1
  Counter-CW (Q,dO):     GPU1→GPU4, GPU2→GPU1, GPU3→GPU2, GPU4→GPU3
```

```
BACKWARD STEP 1: First bi-directional rotation

After rotation, each GPU receives:
  GPU1: receives K4,V4 (from GPU4 clockwise), receives Q2,dO2 (from GPU2)
  GPU2: receives K1,V1 (from GPU1 clockwise), receives Q3,dO3 (from GPU3)
  GPU3: receives K2,V2 (from GPU2 clockwise), receives Q4,dO4 (from GPU4)
  GPU4: receives K3,V3 (from GPU3 clockwise), receives Q1,dO1 (from GPU1)

GPU1 computations:
  For dL/dQ1 (using received K4,V4):
    s14 = Q1 × K4^T / √d = 0.0, A14 ≈ 0.143
    dL/dQ1_from_K4 = [0, 0.712, 0, 0.712]
    dL/dQ1_accum = [0.090, 0.090, 0, 0] + [0, 0.712, 0, 0.712]
                 = [0.090, 0.802, 0, 0.712]

  For dL/dK1, dL/dV1 (using received Q2, dL/dO2):
    s21 = Q2 × K1^T / √d = 0.5, A21 ≈ 0.235
    dL/dK1_from_Q2 = [0, 0.180, 0, 0.180]
    dL/dV1_from_Q2 = [0, 0.235, 0, 0]
    dL/dK1_accum = [0.090, 0, 0.090, 0] + [0, 0.180, 0, 0.180]
                 = [0.090, 0.180, 0.090, 0.180]
    dL/dV1_accum = [0.235, 0, 0, 0] + [0, 0.235, 0, 0]
                 = [0.235, 0.235, 0, 0]

GPU2 computations:
  For dL/dQ2 (using received K1,V1):
    dL/dQ2_from_K1 = [0.180, 0.180, 0, 0]
    dL/dQ2_accum = [0, 0, 0.540, 0.540] + [0.180, 0.180, 0, 0]
                 = [0.180, 0.180, 0.540, 0.540]

  For dL/dK2, dL/dV2 (using received Q3, dL/dO3):
    dL/dK2_accum = [0, 0.540, 0, 0.540] + [0.258, 0.258, 0, 0]
                 = [0.258, 0.798, 0, 0.540]
    dL/dV2_accum = [0, 0.235, 0, 0] + [0, 0, 0.250, 0]
                 = [0, 0.235, 0.250, 0]

GPU3 and GPU4: Similar bi-directional computations...
```

After Step 1 - Progress Summary:

GPU	dL/dQ (K chunks seen)	dL/dK (Q contribs)	dL/dV (Q contribs)
GPU1	K1, K4 (2 of 4)	Q1, Q2 (2 of 4)	Q1, Q2 (2 of 4)
GPU2	K2, K1 (2 of 4)	Q2, Q3 (2 of 4)	Q2, Q3 (2 of 4)
GPU3	K3, K2 (2 of 4)	Q3, Q4 (2 of 4)	Q3, Q4 (2 of 4)
GPU4	K4, K3 (2 of 4)	Q4, Q1 (2 of 4)	Q4, Q1 (2 of 4)

BACKWARD STEPS 2-3: Continue bi-directional rotation

```
Step 2: GPU1 receives K3,V3 (CW) and Q3,dO3 (CCW)
        GPU2 receives K4,V4 (CW) and Q4,dO4 (CCW)
        ...

Step 3: GPU1 receives K2,V2 (CW) and Q4,dO4 (CCW) - FINAL chunk!
        GPU2 receives K3,V3 (CW) and Q1,dO1 (CCW) - FINAL chunk!
        ...
```

After Step 3 (P-1 = 3 rotations): ALL GRADIENTS COMPLETE!

GPU	dL/dQ (K chunks seen)	dL/dK (Q contribs)	dL/dV (Q contribs)
GPU1	K1,K4,K3,K2 (ALL 4) ✓	Q1,Q2,Q3,Q4 (ALL 4) ✓	Q1,Q2,Q3,Q4 (ALL 4) ✓
GPU2	K2,K1,K4,K3 (ALL 4) ✓	Q2,Q3,Q4,Q1 (ALL 4) ✓	Q2,Q3,Q4,Q1 (ALL 4) ✓
GPU3	K3,K2,K1,K4 (ALL 4) ✓	Q3,Q4,Q1,Q2 (ALL 4) ✓	Q3,Q4,Q1,Q2 (ALL 4) ✓
GPU4	K4,K3,K2,K1 (ALL 4) ✓	Q4,Q1,Q2,Q3 (ALL 4) ✓	Q4,Q1,Q2,Q3 (ALL 4) ✓

==========
SECTION 9.8: FINAL BACKWARD STATE
==========

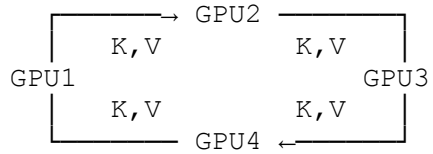
After all 4 steps (P rotations), each GPU has:

FINAL GRADIENTS (Complete)	
GPU1	dL/dQ1 = accumulated from K1, K2, K3, K4 (all K chunks seen) dL/dK1 = accumulated from Q1, Q2, Q3, Q4 (all Q contributions) dL/dV1 = accumulated from Q1, Q2, Q3, Q4 (all Q contributions)
GPU2	dL/dQ2 = accumulated from K1, K2, K3, K4 dL/dK2 = accumulated from Q1, Q2, Q3, Q4 dL/dV2 = accumulated from Q1, Q2, Q3, Q4
GPU3	dL/dQ3 = accumulated from K1, K2, K3, K4 dL/dK3 = accumulated from Q1, Q2, Q3, Q4 dL/dV3 = accumulated from Q1, Q2, Q3, Q4
GPU4	dL/dQ4 = accumulated from K1, K2, K3, K4 dL/dK4 = accumulated from Q1, Q2, Q3, Q4 dL/dV4 = accumulated from Q1, Q2, Q3, Q4

==========
SECTION 9.9: BACKWARD COMMUNICATION DIAGRAM
==========

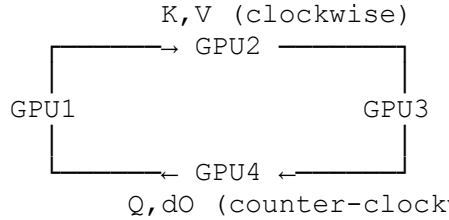
FORWARD vs BACKWARD COMMUNICATION

FORWARD (single direction - rotate K,V clockwise):



Communication: (P-1) × 2 × (s/P) × d ≈ 2 × s × d

BACKWARD (BI-DIRECTIONAL - rotate K,V CW and Q,dO CCW simultaneously):



Communication: 2 × [(P-1) × 2 × (s/P) × d] ≈ 4 × s × d
(K,V rotation) (Q,dO rotation)

Backward / Forward ratio = 2×

WHY BI-DIRECTIONAL IS NEEDED

In forward pass:
  - Q stays local, K,V rotate
  - Each Q_i computes attention over all K,V chunks

In backward pass, we need:

  1. dL/dQ_i = Σ_j (gradient from attending to K_j)
     → Need Q_i to see all K,V chunks (same as forward)
     → Rotate K,V clockwise

  2. dL/dK_i = Σ_j (gradient from Q_j attending to K_i)
     dL/dV_i = Σ_j (gradient from Q_j attending to V_i)
     → Need K_i,V_i to see all Q,dO chunks
     → Rotate Q,dO counter-clockwise (opposite direction)

KEY INSIGHT: Forward and backward have SYMMETRIC but OPPOSITE needs!
  Forward: Q fixed, K,V moves → computes O_i
  Backward for dL/dQ: Q fixed, K,V moves → accumulates dL/dQ_i
  Backward for dL/dK,dL/dV: K,V fixed, Q moves → accumulates dL/dK,dL/dV

==
SECTION 9.10: BACKWARD OVERLAP (Compute-Communication)
==

With bi-directional ring, we can overlap computation with BOTH directions!

Time →
GPU1:
K,V recv: [▒▒ K4,V4 ▒▒][▒▒ K3,V3 ▒▒][▒▒ K2,V2 ▒▒]
Q,dO recv: [▒▒ Q2,dO2 ▒▒][▒▒ Q3,dO3 ▒▒][▒▒ Q4,dO4 ▒▒]
Compute: [██ local ██][██ step1 ██][██ step2 ██][██ step3 ██]
                  ↑ overlap both directions!

Each compute step processes:
  - Received K,V → update dL/dQ
  - Received Q,dO → update dL/dK, dL/dV

==
SECTION 9.11: BACKWARD SUMMARY
==

RING ATTENTION BACKWARD - KEY POINTS

1. dL/dQ: Same pattern as forward
   - Rotate K,V CLOCKWISE
   - Accumulate dL/dQ from each K chunk

2. dL/dK and dL/dV: OPPOSITE pattern
   - Rotate Q,dO COUNTER-CLOCKWISE
   - Accumulate dL/dK, dL/dV from each Q chunk

3. BOTH rotations are needed simultaneously!
   - K,V clockwise (for dL/dQ)
   - Q,dO counter-clockwise (for dL/dK, dL/dV)

4. Communication volume:
   - Forward: $2 \times s \times d$ (K,V only)
   - Backward: $4 \times s \times d$ (K,V + Q,dO)
   - Backward = 2× Forward communication

5. Memory efficiency maintained:
   - Still $O(s^2/P^2)$ for attention matrix chunks
   - Recomputation in backward (don't store full attention)

6. Online softmax backward:
   - Must track m (max) and l (sum) from forward
   - Or recompute during backward

7. SUMMARY TABLE:

Pass	What rotates	Direction	What accumulates
Forward	K, V	Clockwise	O_i (output)
Backward (for dL/dQ)	K, V	Clockwise	dL/dQ_i
Backward (for dL/dK, dL/dV)	Q, dL/dO	Counter-CW	dL/dK_i, dL/dV_i

==========
SECTION 10: MEMORY ANALYSIS
==========

MEMORY COMPARISON

Standard Attention (single GPU):
  Q, K, V: $3 \times s \times d$
  Attention scores (QK^T): $s \times s$ ← THIS IS THE PROBLEM!
  Output: $s \times d$
  Total: $O(s^2 + s \times d)$

```
Ring Attention (P GPUs):
  Q_local: (s/P) × d
  K,V current chunk: 2 × (s/P) × d
  K,V next buffer:   2 × (s/P) × d   (for overlap)
  Local attention scores: (s/P) × (s/P)  ← MUCH SMALLER!
  Partial output: (s/P) × d
  Online softmax state: m, l per row = 2 × (s/P)
  Total per GPU: O(s×d/P + s²/P²)

Memory reduction: From O(s²) to O(s²/P²)!
```

Example: s = 128K, d = 128, P = 8, fp16

Standard:
  Attention matrix: 128K × 128K × 2 bytes = 32 GB ← IMPOSSIBLE!

Ring Attention:
  Local attention: 16K × 16K × 2 bytes = 0.5 GB   ← FITS!
  Per GPU memory: ~1-2 GB for attention

SECTION 11: RING ATTENTION VS ZIG-ZAG ATTENTION

Problem with basic Ring Attention: LOAD IMBALANCE in causal masking!

In causal (autoregressive) attention:
  - Token i can only attend to tokens 0 to i
  - Early tokens have LESS computation than later tokens

CAUSAL MASKING LOAD IMBALANCE

```
With 4 GPUs, 4 tokens (1 per GPU):

Token 0 (GPU1): attends to [0]          → 1 attention computation
Token 1 (GPU2): attends to [0,1]        → 2 attention computations
Token 2 (GPU3): attends to [0,1,2]      → 3 attention computations
Token 3 (GPU4): attends to [0,1,2,3]    → 4 attention computations

GPU1 does 1 unit of work, GPU4 does 4 units → 4× imbalance!
```

ZIG-ZAG ATTENTION Solution:

Instead of assigning consecutive tokens to consecutive GPUs,
use a zig-zag pattern to balance computation:

Standard assignment (tokens per GPU):

```
  GPU1: [0, 1, 2, ...]     (early tokens - less work)
  GPU4: [..., s-3, s-2, s-1] (late tokens - more work)

Zig-Zag assignment:
  GPU1: [0, 7, 8, 15, 16, ...]   (mix of early and late)
  GPU2: [1, 6, 9, 14, 17, ...]
  GPU3: [2, 5, 10, 13, 18, ...]
  GPU4: [3, 4, 11, 12, 19, ...]

Pattern: 0,1,2,3,4,5,6,7 → GPU1,GPU2,GPU3,GPU4,GPU4,GPU3,GPU2,GPU1 (zig-zag)
```

```
┌──────────────────────────────────────────────────────────────────────────┐
│ ZIG-ZAG VISUALIZATION (8 tokens, 4 GPUs)                                 │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│ Token index:  0    1    2    3    4    5    6    7                       │
│               │    │    │    │    │    │    │    │                     │
│               ↓    ↓    ↓    ↓    ↓    ↓    ↓    ↓                     │
│ Assignment:  GPU1 GPU2 GPU3 GPU4 GPU4 GPU3 GPU2 GPU1                     │
│                                                                          │
│ GPU1 gets: tokens 0, 7  → attends to [0] and [0-7]     = 1+8 = 9         │
│ GPU2 gets: tokens 1, 6  → attends to [0-1] and [0-6]   = 2+7 = 9         │
│ GPU3 gets: tokens 2, 5  → attends to [0-2] and [0-5]   = 3+6 = 9         │
│ GPU4 gets: tokens 3, 4  → attends to [0-3] and [0-4]   = 4+5 = 9         │
│                                                                          │
│ PERFECTLY BALANCED!                                                      │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

Comparison Table:

Aspect	Basic Ring Attention	Zig-Zag Ring Attention
Token assignment	Consecutive	Interleaved (zig-zag)
Load balance	Poor (with causal mask)	Perfect
Causal work/GPU	Varies 1x to Px	Balanced (avg)
Implementation	Simpler	More complex indexing
Non-causal	Already balanced	No benefit needed

```
================================================================================
SECTION 12: ALL-GATHER VS RING (ALL-TO-ALL) IMPLEMENTATION
================================================================================

Two main approaches to implement context parallelism:
```

```
┌──────────────────────────────────────────────────────────────────────────┐
│ APPROACH 1: ALL-GATHER IMPLEMENTATION                                    │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
```

```
Strategy: All-gather K,V, then compute full attention locally

Steps:
  1. Each GPU has local Q_i, K_i, V_i (shape: s/P × d)
  2. All-Gather K: K_full = AllGather([K_1, K_2, ..., K_P])  (shape: s × d)
  3. All-Gather V: V_full = AllGather([V_1, V_2, ..., V_P])  (shape: s × d)
  4. Compute: O_i = Attention(Q_i, K_full, V_full)
  5. (No need to gather Q - each GPU only needs its local Q)

Communication:
  - All-Gather K: (P-1) × (s/P × d) = (P-1) × s × d / P
  - All-Gather V: (P-1) × (s/P × d) = (P-1) × s × d / P
  - Total: 2 × (P-1) × s × d / P ≈ 2 × s × d  (for large P)

Memory: Must store full K, V after gather: 2 × s × d per GPU

Pros: Simple implementation, good for small P
Cons: High memory (stores full K,V), all communication upfront
```

```
APPROACH 2: RING (ALL-TO-ALL) IMPLEMENTATION

Strategy: Pass K,V chunks around ring, compute incrementally

Steps:
  1. Each GPU has local Q_i, K_i, V_i (shape: s/P × d)
  2. For step = 0 to P-1:
       a. Compute partial attention with current K,V chunk
       b. Send current K,V to next GPU (non-blocking)
       c. Receive next K,V from previous GPU
       d. Update online softmax accumulators
  3. Final output already computed incrementally

Communication:
  - P-1 send/recv operations of size 2 × (s/P × d) each
  - Total: (P-1) × 2 × s × d / P ≈ 2 × s × d  (same as all-gather!)
  - BUT: distributed over time, overlaps with compute

Memory: Only stores 2 chunks: current + next buffer: 4 × s × d / P

Pros: Low memory, compute-comm overlap
Cons: More complex, requires online softmax
```

Comparison Table:

Aspect	All-Gather	Ring (All-to-All)
Communication volume	2 × s × d	2 × s × d (same)

Communication pattern	Collective, upfront	P2P, distributed
Compute-comm overlap	None	Full overlap possible
Memory per GPU	O(s × d)	O(s × d / P)
K,V memory	Full: 2 × s × d	2 chunks: 4 × s × d / P
Attention matrix mem	(s/P) × s	(s/P) × (s/P)
Implementation	Simple	Complex (online softmax)
Best for	Small P, simple code	Large P, memory constrained
Flash Attention compat	Direct	Requires modification

==
SECTION 13: PRACTICAL CONSIDERATIONS
==

WHEN TO USE CONTEXT PARALLELISM

Use CP when:
- Sequence length is very long (>32K tokens)
- Attention memory exceeds GPU capacity
- Already using TP and need more parallelism
- Have good inter-GPU bandwidth (NVLink recommended)

Don't use CP when:
- Sequence length is moderate (<8K)
- Flash Attention alone is sufficient
- Inter-GPU bandwidth is limited (PCIe bottleneck)

COMBINING WITH OTHER PARALLELISMS

Full 5D Parallelism:
- Data Parallel (DP): split batch
- Tensor Parallel (TP): split hidden dimension
- Pipeline Parallel (PP): split layers
- Sequence Parallel (SP): split sequence in non-attention layers
- Context Parallel (CP): split sequence in attention

Typical configuration for very large models:
- Within node: TP=8 (NVLink), CP=2-4
- Across nodes: PP=4-8, DP=remaining GPUs

CP and SP relationship:

- SP handles: LayerNorm, Dropout, Embeddings
- CP handles: Attention mechanism
- They work together to keep sequence split throughout

==
SECTION 14: COMPLEXITY SUMMARY
==

TIME COMPLEXITY

Standard Attention:
  Compute: $O(s^2 \times d)$
  Communication: 0 (single GPU)

Ring Attention (P GPUs):
  Compute per GPU: $O(s^2 \times d / P)$ (same total, distributed)
  Communication: $O(s \times d)$ per GPU (P-1 sends of $s \times d/P$ each)

  With perfect overlap:
    Time ≈ max(T_compute/P, T_comm)
         ≈ T_compute/P (if compute-bound)

Speedup: Up to P× (limited by communication)

SPACE COMPLEXITY

Standard Attention:
  Q, K, V: $3 \times s \times d$
  Attention matrix: $s \times s$ ← Bottleneck!
  Total: $O(s^2 + s \times d)$

Ring Attention (per GPU):
  Q local: $(s/P) \times d$
  K, V buffers: $4 \times (s/P) \times d$ (2 buffers for overlap)
  Attention matrix: $(s/P) \times (s/P)$ ← P^2 reduction!
  Online state: $2 \times (s/P)$
  Total: $O(s^2/P^2 + s \times d/P)$

Memory reduction: P^2 for attention matrix, P for activations

==
SECTION 15: COMPLETE RING ATTENTION ALGORITHM
==

```python
def ring_attention_forward(Q_local, K_local, V_local, rank, world_size):
    """
    Ring Attention Forward Pass

    Args:
        Q_local: (seq_len/P, d) - local queries
        K_local: (seq_len/P, d) - local keys
        V_local: (seq_len/P, d) - local values
        rank: GPU rank (0 to P-1)
        world_size: P (number of GPUs)

    Returns:
        O_local: (seq_len/P, d) - attention output for local queries
    """
    d = Q_local.shape[-1]
    scale = 1.0 / sqrt(d)

    # Initialize online softmax state
    m = torch.full((Q_local.shape[0],), float('-inf'))  # max scores
    l = torch.zeros((Q_local.shape[0],))                 # sum of exp
    O = torch.zeros_like(Q_local)                        # output accumulator

    # Current K, V chunk (start with local)
    K_curr, V_curr = K_local, V_local
    K_next, V_next = torch.empty_like(K_local), torch.empty_like(V_local)

    prev_rank = (rank - 1) % world_size
    next_rank = (rank + 1) % world_size

    for step in range(world_size):
        # 1. Start async communication (except last step)
        if step < world_size - 1:
            send_req = isend(K_curr, V_curr, dst=next_rank)
            recv_req = irecv(K_next, V_next, src=prev_rank)

        # 2. Compute partial attention with current chunk
        # Scores: (seq_len/P, seq_len/P)
        scores = Q_local @ K_curr.T * scale

        # Online softmax update
        m_new = torch.max(m, scores.max(dim=-1).values)

        # Rescale previous accumulator
        exp_diff = torch.exp(m - m_new)
        l = l * exp_diff
        O = O * exp_diff.unsqueeze(-1)

        # Add new chunk contribution
        exp_scores = torch.exp(scores - m_new.unsqueeze(-1))
        l = l + exp_scores.sum(dim=-1)
        O = O + exp_scores @ V_curr

        m = m_new
```

```
        # 3. Wait for communication and swap buffers
        if step < world_size - 1:
            send_req.wait()
            recv_req.wait()
            K_curr, K_next = K_next, K_curr
            V_curr, V_next = V_next, V_curr

    # Normalize output
    O = O / l.unsqueeze(-1)

    return O
```

==
SECTION 16: SUMMARY
==

KEY TAKEAWAYS
1. Ring Attention enables VERY LONG sequences by: - Splitting sequence across GPUs - Using online softmax to avoid materializing full attention matrix - Overlapping compute and communication
2. Memory savings: - Attention matrix: $O(s^2) \rightarrow O(s^2/P^2)$ - Activations: $O(s) \rightarrow O(s/P)$
3. Communication: - Ring pattern: P2P send/recv in a ring - Volume: $2 \times s \times d$ per GPU (same as all-gather) - But: distributed over time, overlaps with compute
4. Zig-Zag improves load balance for causal attention
5. CP complements SP: - SP: non-attention layers - CP: attention mechanism
6. Backward pass: same ring pattern, reversed direction

==
END OF DOCUMENT
==